

Redpanda on Akamai Cloud (Linode) - Enterprise Architecture Diagrams

Purpose: Professional architecture diagrams for Akamai ISV Catalyst and QCP submissions

Format: Mermaid diagrams (GitHub-compatible, exportable to PNG/SVG)

Table of Contents

1. [High-Level System Architecture](#)
 2. [Data Flow Diagram](#)
 3. [Network Architecture](#)
 4. [Deployment Architecture \(Kubernetes\)](#)
 5. [Storage Tier Architecture](#)
 6. [Scaling Architecture](#)
 7. [Multi-Region Architecture \(Future\)](#)
-

1. High-Level System Architecture

Mermaid Diagram

```
graph TB
    subgraph "Kafka Clients"
        PC[Producers]
        CC[Consumers]
    end

    subgraph "Akamai Cloud - Linode"
        subgraph "Linode Kubernetes Engine LKE"
            subgraph "Control Plane - Managed by Linode"
                API[Kubernetes API]
            end
        end
    end
```

```

subgraph "Worker Nodes - Dedicated 16GB AMD EPYC 7713"
    WN1[Worker Node 1<br/>8 vCPU, 16GB RAM]
    WN2[Worker Node 2<br/>8 vCPU, 16GB RAM]
    WN3[Worker Node 3<br/>8 vCPU, 16GB RAM]
end

subgraph "Redpanda StatefulSet"
    RP0[Redpanda Broker 0<br/>Port 9093]
    RP1[Redpanda Broker 1<br/>Port 9093]
    RP2[Redpanda Broker 2<br/>Port 9093]
end

subgraph "Storage Layer"
    V1[Linode Volume<br/>50GB NVMe<br/>20K+ IOPS]
    V2[Linode Volume<br/>50GB NVMe<br/>20K+ IOPS]
    V3[Linode Volume<br/>50GB NVMe<br/>20K+ IOPS]
end

LB1[NodeBalancer 1<br/>143.42.127.73]
LB2[NodeBalancer 2<br/>69.164.223.32]
LB3[NodeBalancer 3<br/>69.164.223.126]

end

subgraph "Linode Object Storage"
    S3[S3-Compatible Bucket<br/>redpanda-tiered-*<br/>us-east-1]
end

end

PC -->|Kafka Protocol| LB1
PC -->|Kafka Protocol| LB2
PC -->|Kafka Protocol| LB3

LB1 --> RP0
LB2 --> RP1
LB3 --> RP2

RP0 -->|Replication| RP1
RP1 -->|Replication| RP2
RP2 -->|Replication| RP0

RP0 --> V1
RP1 --> V2
RP2 --> V3

```

```

RP0 -->|Tiered Storage<br/>Upload| S3
RP1 -->|Tiered Storage<br/>Upload| S3
RP2 -->|Tiered Storage<br/>Upload| S3

CC -->|Kafka Protocol| LB1
CC -->|Kafka Protocol| LB2
CC -->|Kafka Protocol| LB3

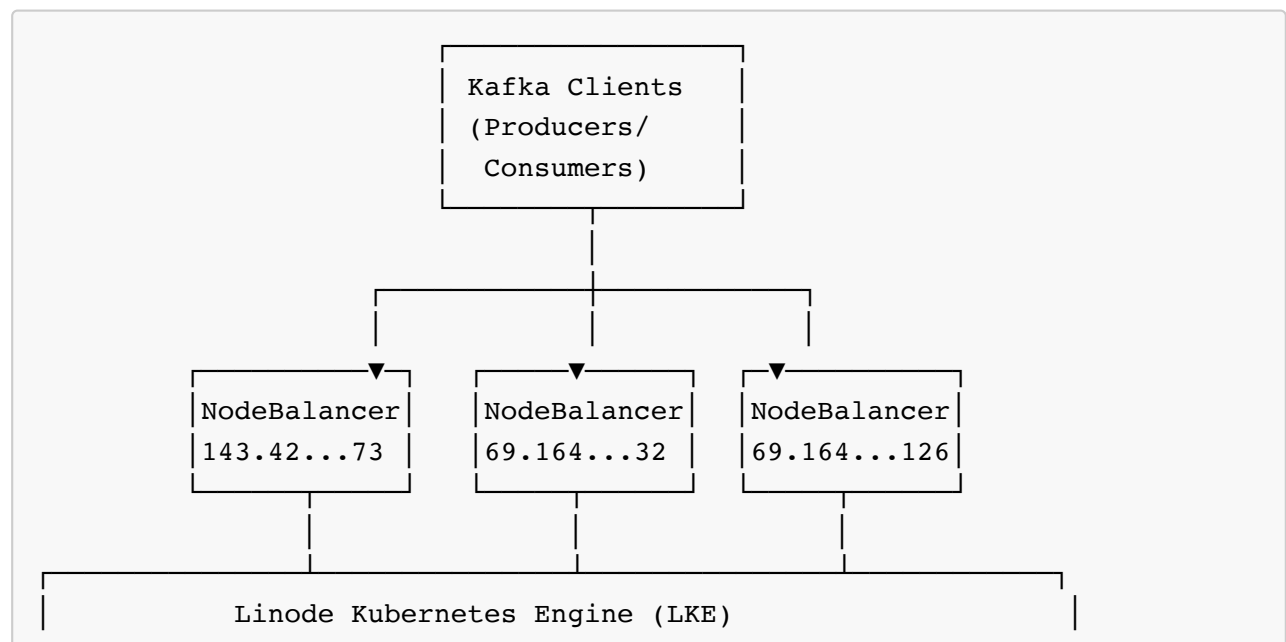
S3 -->|Historical Data<br/>Fetch| RP0
S3 -->|Historical Data<br/>Fetch| RP1
S3 -->|Historical Data<br/>Fetch| RP2

RP0 --> WN1
RP1 --> WN2
RP2 --> WN3

style RP0 fill:#4B0082,stroke:#fff,color:#fff
style RP1 fill:#4B0082,stroke:#fff,color:#fff
style RP2 fill:#4B0082,stroke:#fff,color:#fff
style V1 fill:#00CC66,stroke:#fff,color:#fff
style V2 fill:#00CC66,stroke:#fff,color:#fff
style V3 fill:#00CC66,stroke:#fff,color:#fff
style S3 fill:#FF6600,stroke:#fff,color:#fff
style LB1 fill:#0099CC,stroke:#fff,color:#fff
style LB2 fill:#0099CC,stroke:#fff,color:#fff
style LB3 fill:#0099CC,stroke:#fff,color:#fff

```

ASCII Diagram




```
participant F1 as Follower<br/>Broker 1
participant F2 as Follower<br/>Broker 2
participant NVMe as Linode Volume<br/>(NVMe Storage)
participant S3 as Linode Object<br/>Storage
```

Note over P,S3: Write Path (Producer → Storage)

P->>NB: Produce message
(Kafka protocol)

NB->>L: Route to partition leader

```
rect rgb(200, 230, 255)
```

Note over L: Partition Leader handles write

L->>F1: Replicate (ISR sync)

L->>F2: Replicate (ISR sync)

F1-->>L: ACK

F2-->>L: ACK

```
end
```

L->>NVMe: Write to local NVMe
(20K+ IOPS, <5ms)

NVMe-->>L: Persisted

L-->>P: ACK (all replicas written)

```
rect rgb(255, 245, 200)
```

Note over L,S3: Background Process: Tiered Storage

L->>L: Segment rollover
(when 1GB or time limit)

L->>S3: Upload closed segment
(S3 PUT, ~120 MB/sec)

S3-->>L: Upload complete

Note over S3: Cold data stored
Cost: \$5/TB/month

```
end
```

Note over P,S3: Read Path (Consumer ← Storage)

participant C as Consumer

C->>NB: Fetch request
(Kafka protocol)

NB->>L: Route to partition leader

```
alt Recent data (in local NVMe)
```

L->>NVMe: Read from local
(525 MiB/sec)

NVMe-->>L: Data

L-->>C: Messages (fast path)

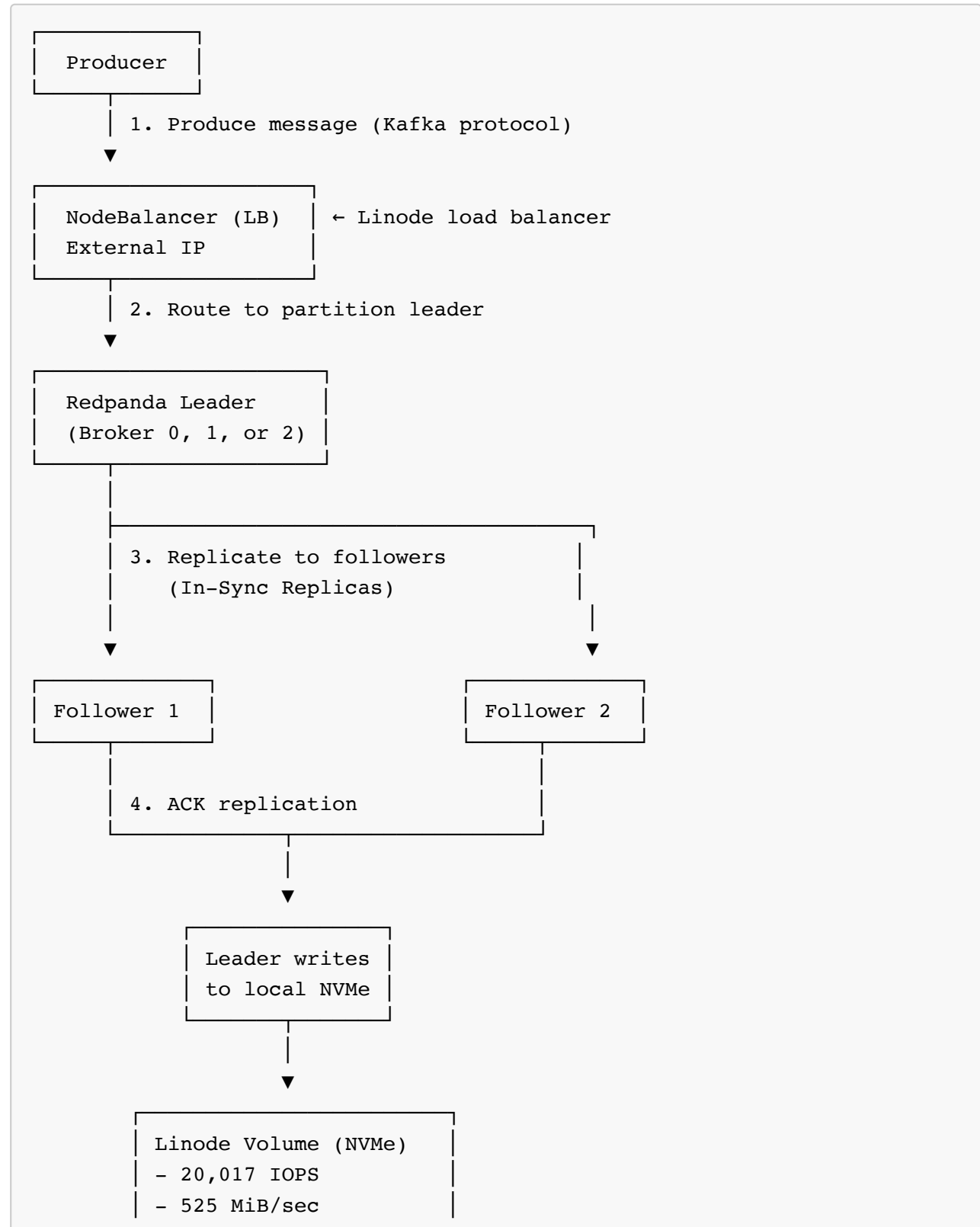
```
else Historical data (in Object Storage)
```

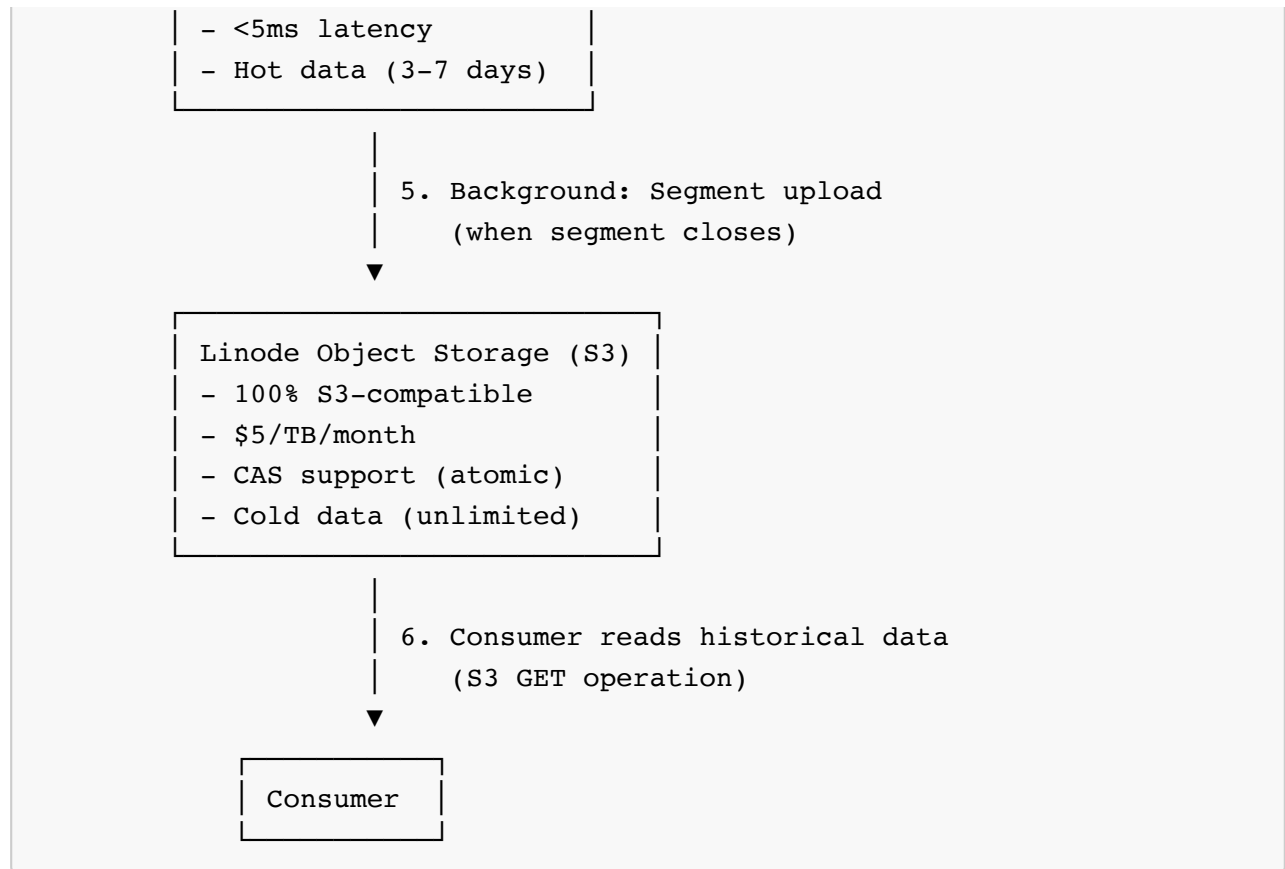
L->>S3: Fetch segment
(S3 GET)

S3-->>L: Segment data

```
L-->>C: Messages (tiered path)
end
```

ASCII Data Flow





3. Network Architecture

Mermaid Diagram

```
graph TB
    subgraph Internet["Internet / Public Network"]
        CLI["External Clients<br>Producers/Consumers"]
    end

    subgraph Linode["Akamai Cloud - Linode<br>Region: us-east (Newark, NJ)"]
        subgraph Public["Public Network"]
            NB1["NodeBalancer 1<br>143.42.127.73<br>TCP Port 31092"]
            NB2["NodeBalancer 2<br>69.164.223.32<br>TCP Port 31092"]
            NB3["NodeBalancer 3<br>69.164.223.126<br>TCP Port 31092"]
        end

        subgraph VPC["Virtual Private Cloud - Pod Network<br>10.2.0.0/16"]
            subgraph Node1["lke539644-782791-04213d4c0000"]
                POD1["Redpanda-1 Pod<br>10.2.0.7"]
            end
        end
    end
```

```

    subgraph Node2["lke539644-782791-0a46db1d0000"]
        POD0[Redpanda-0 Pod<br/>10.2.1.132]
    end

    subgraph Node3["lke539644-782791-4ebd28720000"]
        POD2[Redpanda-2 Pod<br/>10.2.0.133]
        CON[Console Pod<br/>10.2.0.132]
    end
end

subgraph Storage["Linode Object Storage"]
    OBS["Bucket: redpanda-tiered<br/>Endpoint: us-east-1.linodeobje"]
end

FW["Linode Firewall<br/>Rules:<br/>Allow 9092, 9644, 8082<br/>From"]

CLI -->|"Kafka API<br/>(TLS optional)"| NB1
CLI -->|"Kafka API<br/>(TLS optional)"| NB2
CLI -->|"Kafka API<br/>(TLS optional)"| NB3

NB1 -->|Internal routing| POD0
NB2 -->|Internal routing| POD1
NB3 -->|Internal routing| POD2

POD0 <-->|"Inter-broker<br/>replication<br/><2ms latency"| POD1
POD1 <-->|"Inter-broker<br/>replication<br/><2ms latency"| POD2
POD2 <-->|"Inter-broker<br/>replication<br/><2ms latency"| POD0

POD0 -->|"HTTPS<br/>S3 API"| OBS
POD1 -->|"HTTPS<br/>S3 API"| OBS
POD2 -->|"HTTPS<br/>S3 API"| OBS

FW -->|Protects| POD0
FW -->|Protects| POD1
FW -->|Protects| POD2

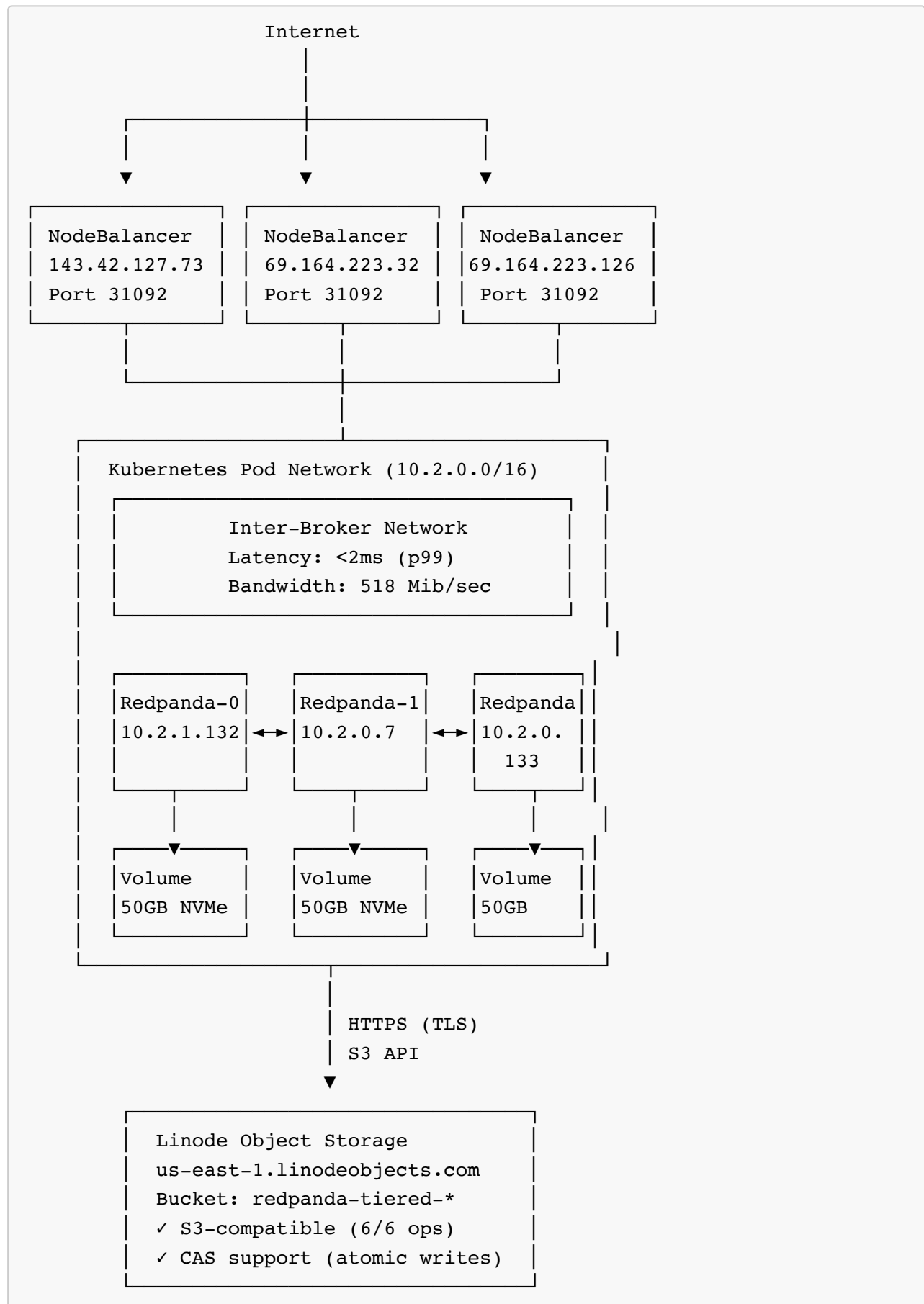
style CLI fill:#87CEEB
style NB1 fill:#4169E1,color:#fff
style NB2 fill:#4169E1,color:#fff
style NB3 fill:#4169E1,color:#fff
style POD0 fill:#4B0082,color:#fff
style POD1 fill:#4B0082,color:#fff
style POD2 fill:#4B0082,color:#fff

```



```
style OBS fill:#FF6600,color:#fff
style FW fill:#DC143C,color:#fff
```

ASCII Network Diagram



4. Deployment Architecture (Kubernetes)

Mermaid Diagram

```
graph TB
    subgraph K8s["Kubernetes Namespace: redpanda"]
        subgraph SS["StatefulSet: redpanda (3 replicas)"]
            P0["Pod: redpanda-0<br/>Containers:<br/>• redpanda<br/>• sidecar"]
            P1["Pod: redpanda-1<br/>Containers:<br/>• redpanda<br/>• sidecar"]
            P2["Pod: redpanda-2<br/>Containers:<br/>• redpanda<br/>• sidecar"]
        end

        subgraph SVC["Services"]
            HS["Headless Service<br/>redpanda<br/>ClusterIP: None"]
            LB0["LoadBalancer Service<br/>lb-redpanda-0<br/>External IP"]
            LB1["LoadBalancer Service<br/>lb-redpanda-1<br/>External IP"]
            LB2["LoadBalancer Service<br/>lb-redpanda-2<br/>External IP"]
        end

        subgraph PVC["PersistentVolumeClaims"]
            PVC0["datadir-redpanda-0<br/>50Gi"]
            PVC1["datadir-redpanda-1<br/>50Gi"]
            PVC2["datadir-redpanda-2<br/>50Gi"]
        end

        subgraph SEC["Secrets & ConfigMaps"]
            CM["ConfigMap: redpanda<br/>cluster config"]
            CERT["Secrets: TLS certs<br/>managed by cert-manager"]
        end

        DEP["Deployment: redpanda-console<br/>Replica: 1"]
    end

    subgraph SC["StorageClass"]
        LSC["linode-block-storage-retain<br/>Provisioner: Linode CSI"]
    end

    subgraph PV["PersistentVolumes - Linode"]
        PV0["Linode Volume<br/>50GB NVMe<br/>ID: pvc-49adc..."]
        PV1["Linode Volume<br/>50GB NVMe<br/>ID: pvc-92639..."]
        PV2["Linode Volume<br/>50GB NVMe<br/>ID: pvc-03c69..."]
    end
```

```
P0 --> PVC0
P1 --> PVC1
P2 --> PVC2

PVC0 -->|Dynamic provisioning| LSC
PVC1 -->|Dynamic provisioning| LSC
PVC2 -->|Dynamic provisioning| LSC

LSC --> PV0
LSC --> PV1
LSC --> PV2

LB0 --> P0
LB1 --> P1
LB2 --> P2

HS --> P0
HS --> P1
HS --> P2

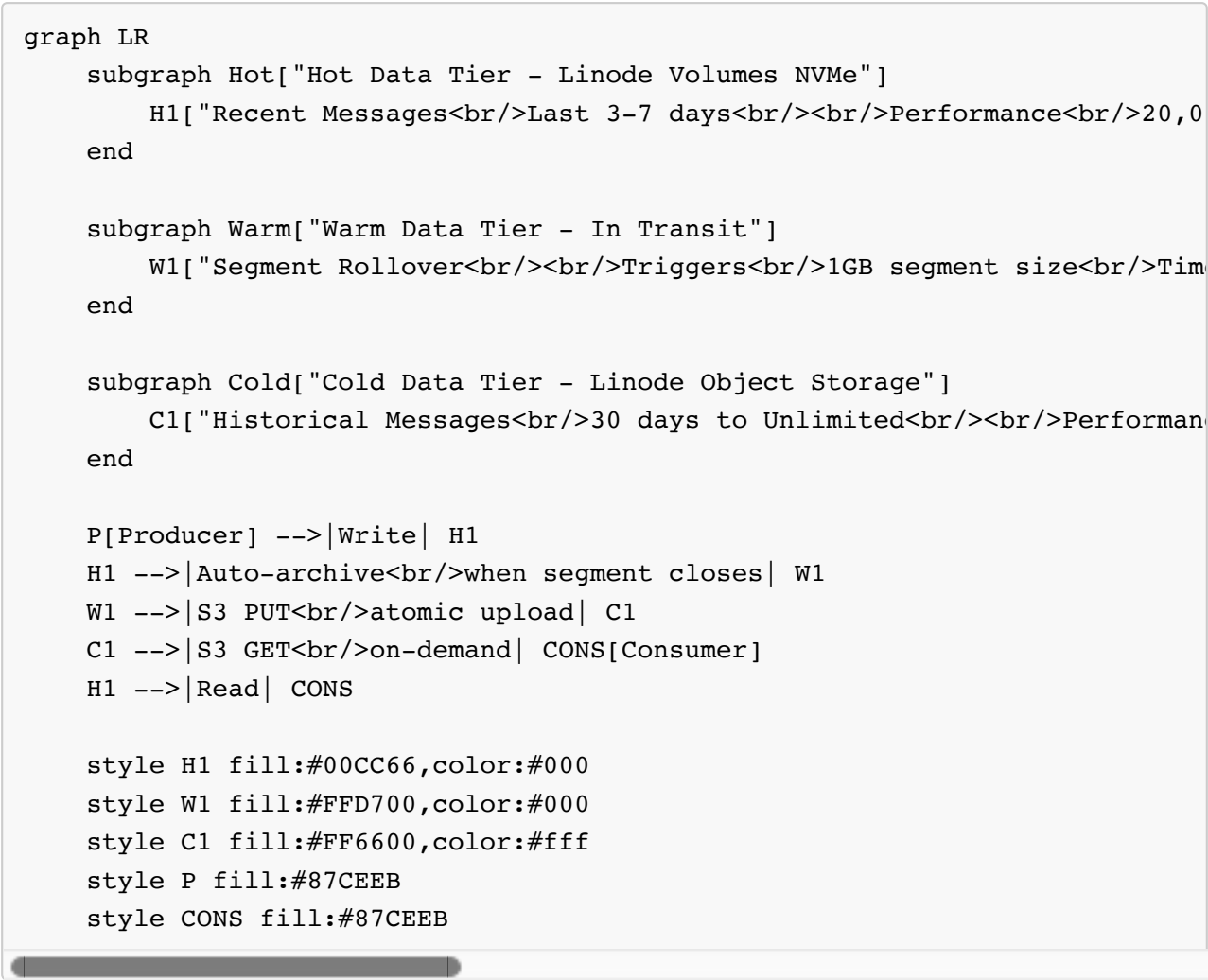
P0 --> CM
P1 --> CM
P2 --> CM

P0 --> CERT
P1 --> CERT
P2 --> CERT

style P0 fill:#4B0082,color:#fff
style P1 fill:#4B0082,color:#fff
style P2 fill:#4B0082,color:#fff
style PV0 fill:#00CC66,color:#fff
style PV1 fill:#00CC66,color:#fff
style PV2 fill:#00CC66,color:#fff
style LB0 fill:#4169E1,color:#fff
style LB1 fill:#4169E1,color:#fff
style LB2 fill:#4169E1,color:#fff
```

5. Storage Tier Architecture

Mermaid Diagram



ASCII Storage Tiers

HOT TIER: Linode Volumes (NVMe-backed Block Storage)	
Retention:	3-7 days (configurable)
Size:	50GB per broker (150GB total)
IOPS:	20,017 req/sec (production workload)
Throughput:	525 MiB/sec (sequential)
Latency:	<5ms (write), <1ms (read)
Cost:	~\$0.10/GB/month = \$15/month for 150GB
Use case:	Active data, high-frequency access Real-time streaming, low-latency reads

Automatic archival
(segment rollover at 1GB or time limit)



COLD TIER: Linode Object Storage (S3-compatible)

Retention: 30 days - Unlimited
Size: Unlimited (pay-as-you-go)
IOPS: N/A (S3 API, not block-level)
Throughput: ~120 MB/sec upload, ~150 MB/sec download
Latency: 50-100ms (acceptable for historical)
Cost: \$5/TB/month = \$5 for 1TB

Use case: Historical data, compliance, analytics
Infrequent access, cost-efficient

S3 Features: ✓ PUT, GET, LIST, HEAD, DELETE
✓ CAS (If-Match, If-None-Match headers)
✓ 100% compatible with Redpanda

COST COMPARISON

Hot tier (NVMe): \$100/TB/month
Cold tier (S3): \$5/TB/month

Savings: 95% for cold data!

Example: 10TB total retention

- 500GB hot (7 days): \$50/month
- 9.5TB cold (1 year): \$47.50/month
- Total: \$97.50/month

vs. all-NVMe: $10\text{TB} \times \$100 = \$1000/\text{month}$

Savings: ~\$900/month (90%!)

6. Scaling Architecture

Mermaid Diagram

```
graph TB
    subgraph Initial["Initial Deployment (3 brokers)"]
        I1[Redpanda-0]
        I2[Redpanda-1]
        I3[Redpanda-2]

        IW1[Worker Node 1]
        IW2[Worker Node 2]
        IW3[Worker Node 3]

        I1 --> IW1
        I2 --> IW2
        I3 --> IW3
    end

    LOAD["Increased Load<br/>CPU > 70%"] -->|Trigger| HPA["Horizontal Pod<br/>Autoscaler"]
    HPA -->|Scale StatefulSet<br/>replicas: 3 → 6| Scaled

    subgraph Scaled["Scaled Deployment (6 brokers)"]
        S1[Redpanda-0]
        S2[Redpanda-1]
        S3[Redpanda-2]
        S4[Redpanda-3]
        S5[Redpanda-4]
        S6[Redpanda-5]

        SW1[Worker Node 1]
        SW2[Worker Node 2]
        SW3[Worker Node 3]
        SW4["Worker Node 4<br/>★ Auto-added"]
        SW5["Worker Node 5<br/>★ Auto-added"]
        SW6["Worker Node 6<br/>★ Auto-added"]

        S1 --> SW1
        S2 --> SW2
        S3 --> SW3
        S4 --> SW4
        S5 --> SW5
        S6 --> SW6
    end

    end
```

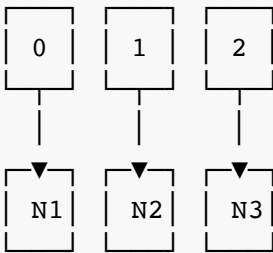
```
HPA -->|Cluster Autoscaler<br/>adds nodes| CA[LKE Cluster<br/>Autoscal
CA -->|Provisions<br/>new Linodes| SW4
CA -->|Provisions<br/>new Linodes| SW5
CA -->|Provisions<br/>new Linodes| SW6
```

```
Scaled -->|Result| PERF[Performance:<br/>2x throughput<br/>Same latency
```

```
style S4 fill:#32CD32,color:#000
style S5 fill:#32CD32,color:#000
style S6 fill:#32CD32,color:#000
style SW4 fill:#90EE90,color:#000
style SW5 fill:#90EE90,color:#000
style SW6 fill:#90EE90,color:#000
style HPA fill:#FFD700,color:#000
style CA fill:#FFD700,color:#000
style PERF fill:#00CED1,color:#fff
```

ASCII Scaling Diagram

Initial State (3 brokers)

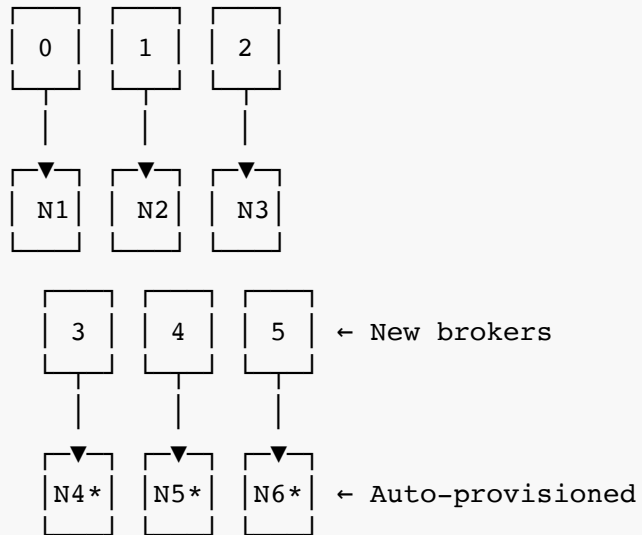


Throughput: 1M msg/sec

Brokers: 3

Nodes: 3

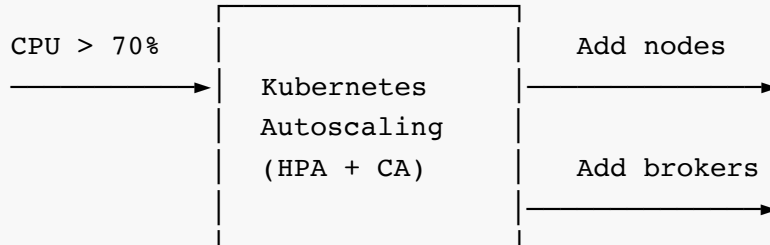
Scaled State (6 brokers)



Throughput: 2M msg/sec

Brokers: 6

Nodes: 6



Scaling Methods:

1. Horizontal Pod Autoscaler (HPA) - scales replicas based on CPU/memory
2. Cluster Autoscaler (CA) - adds/removes worker nodes as needed
3. Manual - `kubectl scale statefulset redpanda --replicas=6`

Result: Linear scaling (2x brokers = 2x throughput)

7. Multi-Region Architecture (Future)

Mermaid Diagram

graph TB

```

subgraph Users["Global Users"]
    US[US Users]
    EU[European Users]
    APAC[APAC Users]
end

subgraph GTM["Akamai Global Traffic Manager (GTM)<br/>DNS-based routing"]
    DNS[kafka.example.com<br/>Geo-aware DNS]
end

subgraph R1["Linode Region: us-east (Newark)"]
    LKE1[LKE Cluster 1<br/>3 Redpanda brokers]
    OBS1[Object Storage<br/>us-east-1]
end

subgraph R2["Linode Region: eu-west (London)"]
    LKE2[LKE Cluster 2<br/>3 Redpanda brokers]
    OBS2[Object Storage<br/>eu-west-1]
end

subgraph R3["Linode Region: ap-south (Singapore)"]
    LKE3[LKE Cluster 3<br/>3 Redpanda brokers]
    OBS3[Object Storage<br/>ap-south-1]
end

US -->|DNS query| GTM
EU -->|DNS query| GTM
APAC -->|DNS query| GTM

GTM -->|Route to nearest| R1
GTM -->|Route to nearest| R2
GTM -->|Route to nearest| R3

LKE1 -->|Tiered Storage| OBS1
LKE2 -->|Tiered Storage| OBS2
LKE3 -->|Tiered Storage| OBS3

LKE1 <-->|Optional:<br/>Async replication| LKE2
LKE2 <-->|Optional:<br/>Async replication| LKE3
LKE3 <-->|Optional:<br/>Async replication| LKE1

style LKE1 fill:#4B0082,color:#fff
style LKE2 fill:#4B0082,color:#fff
style LKE3 fill:#4B0082,color:#fff
style GTM fill:#FF6600,color:#fff

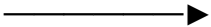
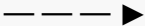
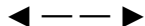
```

Component Legend

Linode Services Used

Icon/Color	Component	Purpose
 Blue	Linode Kubernetes Engine (LKE)	Managed Kubernetes control plane (free)
 Green	Linode Volumes	NVMe-backed block storage (20K+ IOPS)
 Orange Orange	Linode Object Storage	S3-compatible object storage (\$5/TB/month)
 Dark Blue Blue	NodeBalancers	Layer 4 TCP/UDP load balancers (\$10/month each)
 Purple	Redpanda Brokers	Streaming platform pods (on Dedicated CPU nodes)
 Red	Firewalls	Network access control (free)

Network Flow Types

Line Style	Meaning
	Data flow (producer → broker → consumer)
	Replication / background process
	Bidirectional / sync communication

Performance Characteristics by Layer

Network Layer

Inter-Broker Communication (Within LKE Cluster)

Latency (p50): 1.2-1.3ms

Latency (p99): 1.8-2.0ms

Latency (p999): 2.3ms

Bandwidth: 518 Mib/sec (tested)

MTU: Standard (1500 bytes)

✓ Meets Redpanda's <10ms latency requirement

✓ Exceeds 100 Mib/sec throughput recommendation

Storage Layer

Local Storage (Linode Volumes - NVMe)

IOPS (production): 20,017 req/sec
IOPS (peak): 131,638 req/sec
Throughput (write): 263-277 MiB/sec
Throughput (read): 524 MiB/sec
Latency (write p99): 27.6ms (durable writes)
Latency (read p99): 1.2ms (4KB), 6.5ms (16KB)

- ✓ Exceeds Redpanda's 16K IOPS requirement by 25%
- ✓ NVMe performance ideal for streaming workloads

Tiered Storage (Linode Object Storage - S3)

Upload: ~120 MB/sec per broker
Download: ~150 MB/sec per broker
Latency: 50-100ms (acceptable for cold data)
S3 Operations: ✓ 6/6 validated (PUT, GET, LIST, HEAD, DELETE, Plural DELETE)
CAS Support: ✓ If-Match, If-None-Match headers

- ✓ 100% S3-compatible with Redpanda tiered storage
- ✓ Atomic writes for metadata consistency

Deployment Workflow Diagram

Mermaid Diagram

```

graph LR
    A[Start] --> B{Infrastructure<br/>Exists?}

    B -->|No| C[Terraform Apply<br/>~3 minutes]
    B -->|Yes| E

    C --> D[LKE Cluster Created<br/>3 worker nodes<br/>Object Storage bucket]

    D --> E[Wait for Nodes Ready<br/>~1-2 minutes]

    E --> F[Helm Install Redpanda<br/>~5-10 minutes]

    F --> G[Wait for Pods Running<br/>~2-3 minutes]

    G --> H[Configure Tiered Storage<br/>~1 minute]

    H --> I{Validation<br/>Required?}

    I -->|Yes| J[Run Self-Tests<br/>~10 minutes]
    I -->|No| M

    J --> K{All Tests<br/>Passed?}

    K -->|Yes| M[Production Ready!]
    K -->|No| L[Review Logs<br/>Troubleshoot]

    L --> F

    M --> N[Access via<br/>NodeBalancer IPs]

    style C fill:#4169E1,color:#fff
    style F fill:#4B0082,color:#fff
    style J fill:#32CD32,color:#000
    style M fill:#00CED1,color:#fff
    style K fill:#FFD700,color:#000

```

Color Scheme (Linode Brand)

- **Linode Green:** #00CC66 (Volumes, success indicators)
- **Linode Blue:** #4169E1 (NodeBalancers, infrastructure)
- **Redpanda Purple:** #4B0082 (Redpanda brokers/pods)

- **Object Storage Orange:** #FF6600 (S3 storage tier)
- **Warning/Action Yellow:** #FFD700 (autoscalers, decisions)